

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE TEORIA

INFORMACIÓN BÁSICA					
ASIGNATURA:	FISICA COMPUTACIONAL				
TÍTULO DE LA PRÁCTICA:	Resolución de la Práctica3: Problemas 3 cuerpos: Laboratorio B				
NÚMERO DE LA PRÁCTICA:	03	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN:	15/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s):	- Jara Mamani Mariel Alisson			NOTA (0 - 20):	Nota colocada por el docente
DOCENTE:	Prof. Edwin Agapito Llamoca Requena				

RESULTADOS Y PRUEBAS
1 SOLUCIÓN DE LOS EJERCICIOS PROPUESTOS 1.1 Ejercicio 1: Primera Circunferencia de radio 1 Problema: Graficar una circunferencia con radio $r = 1$ cuyo centro esta en el punto $(+b, -c)$. Elegir b y c a su criterio. Implementación: Comenzamos definiendo las variables y condiciones iniciales, los valores elegidos para b y c son 3 y 2 respectivamente. Pasamos a definir la funciones de aceleración a_x y a_y. El paso de tiempo es $h = 0.1$ y $k = 0.1$, empezaremos dibujando el círculo y luego a traves de un for haremos las trayectorias. Código fuente: exe1.py <pre>import matplotlib.pyplot as plt import numpy as np # Datos iniciales h = 0.1 k = 0.1 tfin = 30.0 r = 1.0 b1, c1 = 3, -2 plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8}) plt.figure(figsize=(12, 6)) # Dibujar la circunferencia theta = np.linspace(0, 2 * np.pi, 100) x_circ = b1 + np.cos(theta) y_circ = c1 + np.sin(theta) plt.plot(x_circ, y_circ, "black", linewidth=2, label=f"Masa1 en ({b1}, {c1})") # Funciones de aceleración def ax(x, y): if x == 0 and y == 0: return 0 return -x / ((x**2 + y**2) ** (3 / 2)) def ay(x, y): if x == 0 and y == 0: return 0 return -y / ((x**2 + y**2) ** (3 / 2))</pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

for vx0 in np.arange(0.2, 1.5, k):
    x, y = 0.0, -4.0
    vx, vy = vx0, 0.0

    px = [x]
    py = [y]

    for t in np.arange(0, tfin, h):
        vx += ax(x, y) * h
        vy += ay(x, y) * h

        x += vx * h
        y += vy * h

        px.append(x)
        py.append(y)

    plt.plot(px, py)

# Configuración de la gráfica
plt.title("Laboratorio 3 - Ejercicio 1")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.legend()
plt.grid(False)
plt.gca().set_aspect("equal", adjustable="datalim")
plt.savefig("img/pr_b/exel.png", dpi=300, bbox_inches="tight")
plt.show()

```

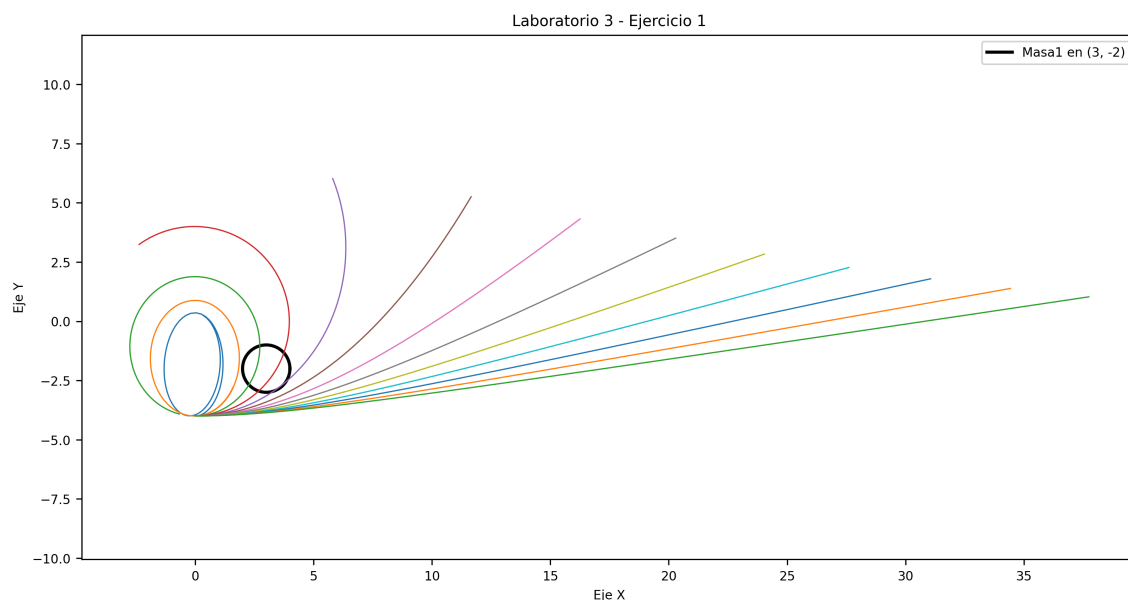


Figura 1: Gráfico de la primera circunferencia con radio $r = 1$ m

Resultados: El gráfico muestra trayectorias de partículas que se originan desde el punto $(0, -4)$ con diferentes velocidades iniciales. Como se ve, las trayectorias más lentas (v_x pequeño) orbitan cerca de la masa central, formando bucles cerrados alrededor del punto $(3, -2)$. Conforme v_x aumenta, las órbitas se abren más, evidenciando que la energía cinética inicial determina el tipo de órbita. La circunferencia negra de radio 1 centrada en $(3, -2)$ representa la masa atractora. Todas las trayectorias divergen hacia la derecha cuando tienen suficiente energía, mostrando el comportamiento característico de órbitas parabólicas e hiperbólicas.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

1.2 Ejercicio 2: Segunda Circunferencia de radio 1

Problema: Graficar una circunferencia con radio $r = 1$ cuyo centro esta en el punto $(-b, -c)$.

Implementación: Este ejercicio extiende el anterior al agregar una segunda masa puntual en $(-3, -2)$, manteniendo la primera en $(3, -2)$. Se mantienen las funciones a_x y a_y . De igual forma dibujamos las dos circunferencias y luego a traves de un for haremos las trayectorias.

Código fuente: exe2.py

```
import matplotlib.pyplot as plt
import numpy as np

# Datos iniciales
r = 1.0
h = 0.1
k = 0.1
tfin = 30.0

plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8})
plt.figure(figsize=(12, 6))

# Circunferencia 1
b1, c1 = 3, -2
theta = np.linspace(0, 2 * np.pi, 100)
x_circ = b1 + np.cos(theta)
y_circ = c1 + np.sin(theta)
plt.plot(x_circ, y_circ, "black", linewidth=2, label=f"Masa1 en ({b1}, {c1})")

# Circunferencia 2
b2, c2 = -3, -2
x_circ2 = b2 + np.cos(theta)
y_circ2 = c2 + np.sin(theta)
plt.plot(x_circ2, y_circ2, "black", linewidth=2, label=f"Masa2 en ({b2}, {c2})")

# Funciones de aceleración
def ax(x, y):
    if x == 0 and y == 0:
        return 0
    return -x / ((x**2 + y**2) ** (3 / 2))

def ay(x, y):
    if x == 0 and y == 0:
        return 0
    return -y / ((x**2 + y**2) ** (3 / 2))

for vx0 in np.arange(0.2, 1.5, k):
    x, y = 0.0, -4.0
    vx, vy = vx0, 0.0

    px = [x]
    py = [y]

    for t in np.arange(0, tfin, h):
        vx += ax(x, y) * h
        vy += ay(x, y) * h

        x += vx * h
        y += vy * h

        px.append(x)
        py.append(y)

    plt.plot(px, py)

# Configuración de la gráfica
plt.title("Laboratorio 3 - Ejercicio 2")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```
plt.grid(False)
plt.legend()
plt.gca().set_aspect("equal", adjustable="datalim")
plt.savefig("img/pr_b/exe2.png", dpi=300, bbox_inches="tight")
plt.show()
```

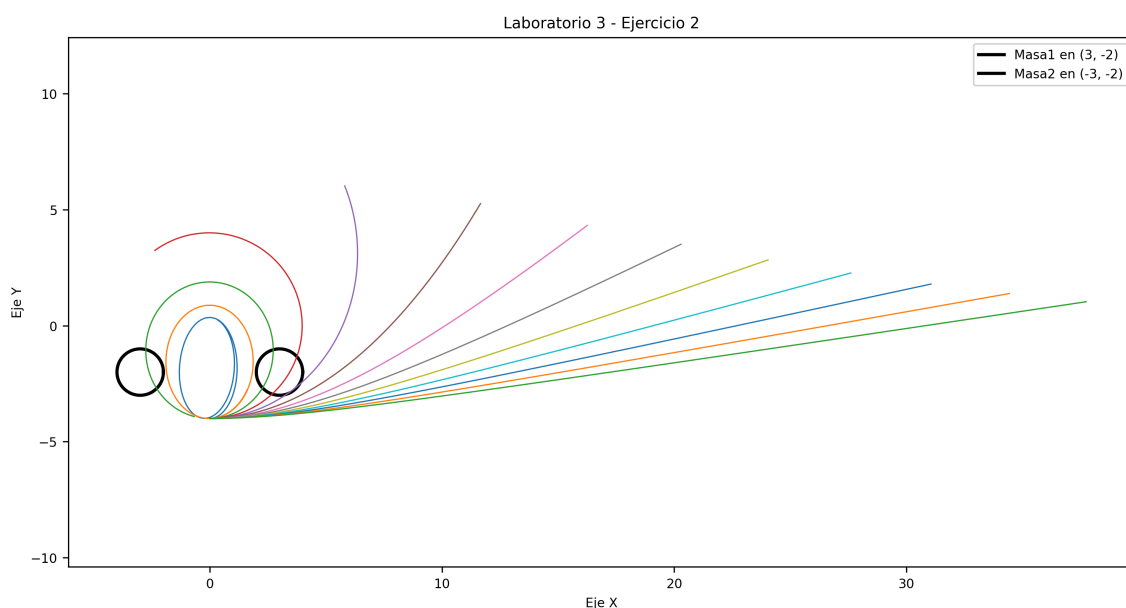


Figura 2: Gráfico de la segunda circunferencia con radio $r = 1 \text{ m}$

Resultados: El gráfico muestra trayectorias en un campo gravitacional de dos masas. Como se ve, las órbitas ahora son asimétricas respecto al eje vertical, favoreciendo trayectorias hacia la derecha donde hay mayor energía potencial combinada. Se observan órbitas complejas alrededor del punto de equilibrio entre las dos masas. Las trayectorias con velocidades bajas permanecen confinadas entre ambas masas, mientras que las de mayor velocidad escapan del sistema. Las dos circunferencias negras en $(3, -2)$ y $(-3, -2)$ marcan los centros de las masas atractoras, haciendo evidente la simetría bilateral del potencial gravitacional.

1.3 Ejercicio 3: Implementar las funciones a_x y a_y

Problema: Genere las funciones $a_x(x, y)$ y $a_y(x, y)$, sabiendo que:

Implementación: Para empezar a implementar primero haremos las funciones de aceleración para un sistema de dos cuerpos de acuerdo a la imagen mostrada del laboratorio.

$$a_x(x, y, b_1, c_1, b_2, c_2) = -\frac{x - b_1}{((x - b_1)^2 + (y - c_1)^2)^{\frac{3}{2}}} - \frac{x - b_2}{((x - b_2)^2 + (y - c_2)^2)^{\frac{3}{2}}}$$

$$a_y(x, y, b_1, c_1, b_2, c_2) = -\frac{y - c_1}{((x - b_1)^2 + (y - c_1)^2)^{\frac{3}{2}}} - \frac{y - c_2}{((x - b_2)^2 + (y - c_2)^2)^{\frac{3}{2}}}$$

Definidas estas funciones, procederemos a mejorar los parámetros de simulación: paso de tiempo $h = 0.3$, rango de velocidades $v_x \in [0.5, 1.5]$, y tiempo final $t_{\text{fin}} = 300$ segundos. Se agrega protección contra escape ilimitado deteniendo la simulación si la partícula alcanza $|x| > 20$ o $|y| > 20$.

Código fuente: exe3.py

```
import matplotlib.pyplot as plt
import numpy as np
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```
plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8})
plt.figure(figsize=(8, 8))

# Datos iniciales
r = 1.0
h = 0.3
k = 0.3
tfin = 300

# Circunferencia 1
b1, c1 = 3, -2

theta = np.linspace(0, 2 * np.pi, 100)
x_circ = b1 + np.cos(theta)
y_circ = c1 + np.sin(theta)
plt.plot(x_circ, y_circ, "black", linewidth=2, label=f"Masa 1 en ({b1}, {c1})")

# Circunferencia 2
b2, c2 = -3, -2
x_circ2 = b2 + np.cos(theta)
y_circ2 = c2 + np.sin(theta)
plt.plot(x_circ2, y_circ2, "black", linewidth=2, label=f"Masa 2 en ({b2}, {c2})")

def ax(x, y, xc1, yc1, xc2, yc2):
    # Masa 1: Distancias y Radio
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    # Masa 2:
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)
    if den1 == 0 or den2 == 0:
        return 0.0

    return (-(x - xc1) / den1) - ((x - xc2) / den2)

def ay(x, y, xc1, yc1, xc2, yc2):
    # Masa 1: Distancias y Radio
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    # Masa 2: Distancias y Radio
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)

    if den1 == 0 or den2 == 0:
        return 0.0
    return (-(y - yc1) / den1) - ((y - yc2) / den2)

for vx0 in np.arange(0.5, 1.5, k):
    x, y = 0.0, -4.0
    vx, vy = vx0, 0.0

    px = [x]
    py = [y]

    for t in np.arange(0, tfin, h):
        vx += ax(x, y, b1, c1, b2, c2) * h
        vy += ay(x, y, b1, c1, b2, c2) * h
        x += vx * h
        y += vy * h

        px.append(x)
        py.append(y)

        if abs(x) > 20 or abs(y) > 20:
            break

    plt.plot(px, py)

# Configuración de la gráfica
plt.title("Laboratorio 3 - Ejercicio 3")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
```

```
plt.grid(False)
plt.legend()
plt.gca().set_aspect("equal", adjustable="datalim")
plt.savefig("img/pr_b/exe3.png", dpi=300, bbox_inches="tight")
plt.show()
```

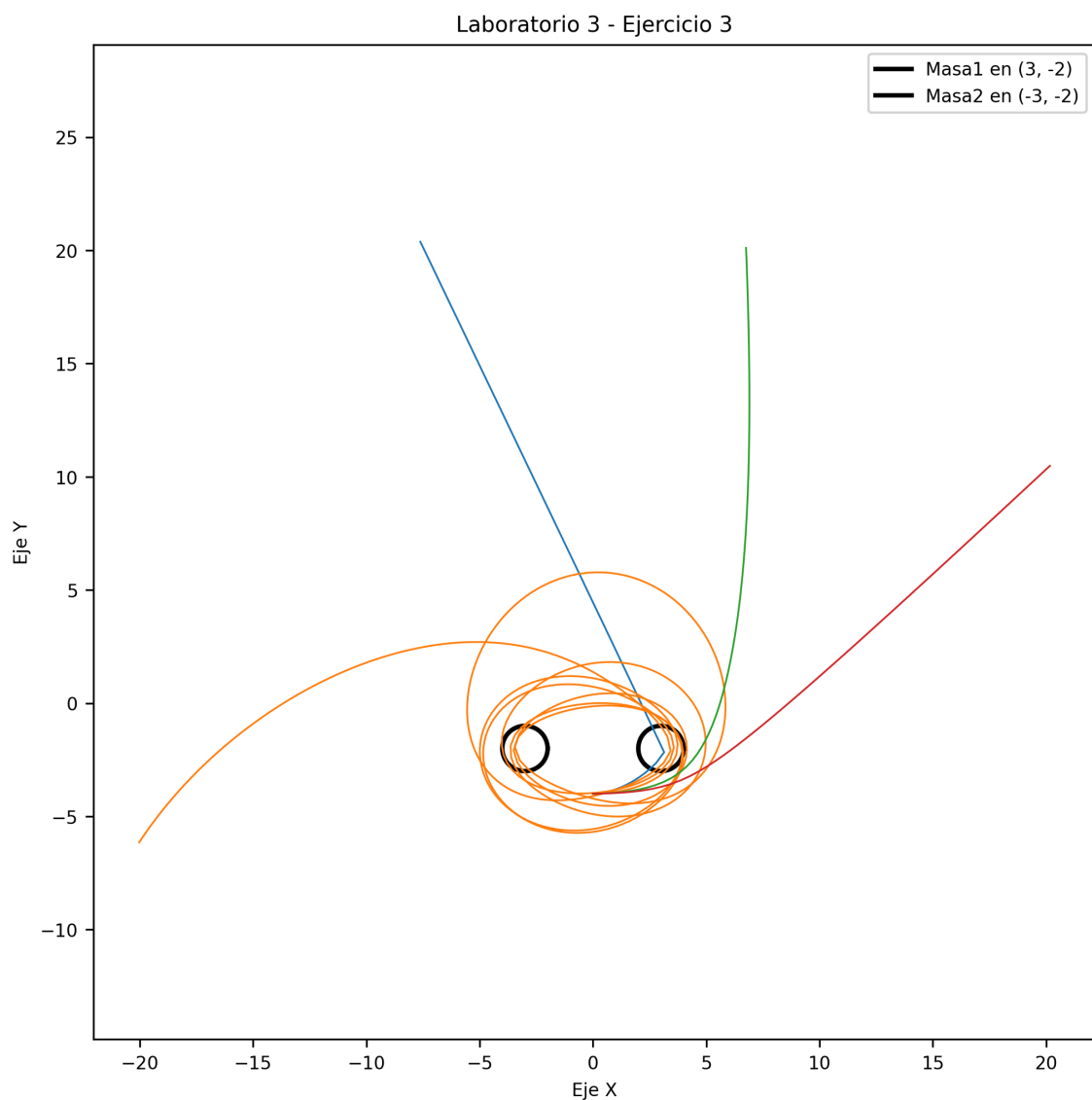


Figura 3: Gráfico de las funciones a_x y a_y

Resultados: El gráfico muestra órbitas más complejas alrededor de dos masas. Como se ve, se forman regiones de caoticidad donde pequeñas variaciones en las condiciones iniciales generan trayectorias radicalmente diferentes. Las órbitas oscilan en patrones intrincados entre las dos masas, con bucles múltiples y ciclos de mayor amplitud. Se observan órbitas periódicas y cuasiperiódicas que forman estructuras roseta alrededor del sistema de dos masas en $(3, -2)$ y $(-3, -2)$. Las trayectorias en color naranja que oscilan en el centro demuestran movimiento altamente acoplado donde ambas masas influyen significativamente en la dinámica.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

1.4 Ejercicio 4: No graficar si la partícula colapsa o se escapa

Problema: Si las trayectorias llegan a las circunferencias, que no grafique

Implementación: Se implementa un sistema de validación de trayectorias que detecta dos condiciones de terminación: colapso y escape. Se añade una condición de colapso verificando si la distancia de la partícula a cualquiera de las dos masas es menor o igual al radio $r = 1$. También se limita el escape lateral con $y > 50$. Solo se grafican trayectorias que ni colapsan ni escapan, permitiendo así visualizar únicamente órbitas ligadas y estables. El paso de tiempo es $h = 0.3$ con $t_{fin} = 100$ segundos y paso de velocidad $k = 0.02$.

Código fuente: exe4.py

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8})
plt.figure(figsize=(10, 8))

# Datos iniciales
r = 1.0
h = 0.3
k = 0.02
tfin = 100

# Circunferencia 1
b1, c1 = 3, -2

theta = np.linspace(0, 2 * np.pi, 100)
x_circ = b1 + np.cos(theta)
y_circ = c1 + np.sin(theta)
plt.plot(x_circ, y_circ, "black", linewidth=2, label=f"Masa 1 en ({b1}, {c1})")

# Circunferencia 2
b2, c2 = -3, -2
x_circ2 = b2 + np.cos(theta)
y_circ2 = c2 + np.sin(theta)
plt.plot(x_circ2, y_circ2, "black", linewidth=2, label=f"Masa 2 en ({b2}, {c2})")

def ax(x, y, xc1, yc1, xc2, yc2):
    # Masa 1: Distancias y Radio
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    # Masa 2:
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)
    if den1 == 0 or den2 == 0:
        return 0.0

    return (-(x - xc1) / den1) - ((x - xc2) / den2)

def ay(x, y, xc1, yc1, xc2, yc2):
    # Masa 1: Distancias y Radio
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    # Masa 2: Distancias y Radio
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)

    if den1 == 0 or den2 == 0:
        return 0.0
    return (-(y - yc1) / den1) - ((y - yc2) / den2)

for vx0 in np.arange(0.5, 1.5, k):
    x, y = 0.0, -4.0
    vx, vy = vx0, 0

    px = [x]
    py = [y]

    for t in np.arange(0, tfin, h):
        vx += ax(x, y, b1, c1, b2, c2) * h
        vy += ay(x, y, b1, c1, b2, c2) * h
```

```
x += vx * h
y += vy * h

if (
    np.sqrt((x - b1) ** 2 + (y - c1) ** 2) <= r
    or np.sqrt((x - b2) ** 2 + (y - c2) ** 2) <= r
):
    break
if y > 50:
    break

px.append(x)
py.append(y)

plt.plot(px, py)

# Configuración de la gráfica
plt.title("Laboratorio 3 - Ejercicio 4")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.grid(False)
plt.legend()
plt.gca().set_aspect("equal", adjustable="datalim")
plt.savefig("img/pr_b/exe4.png", dpi=300, bbox_inches="tight")
plt.show()
```

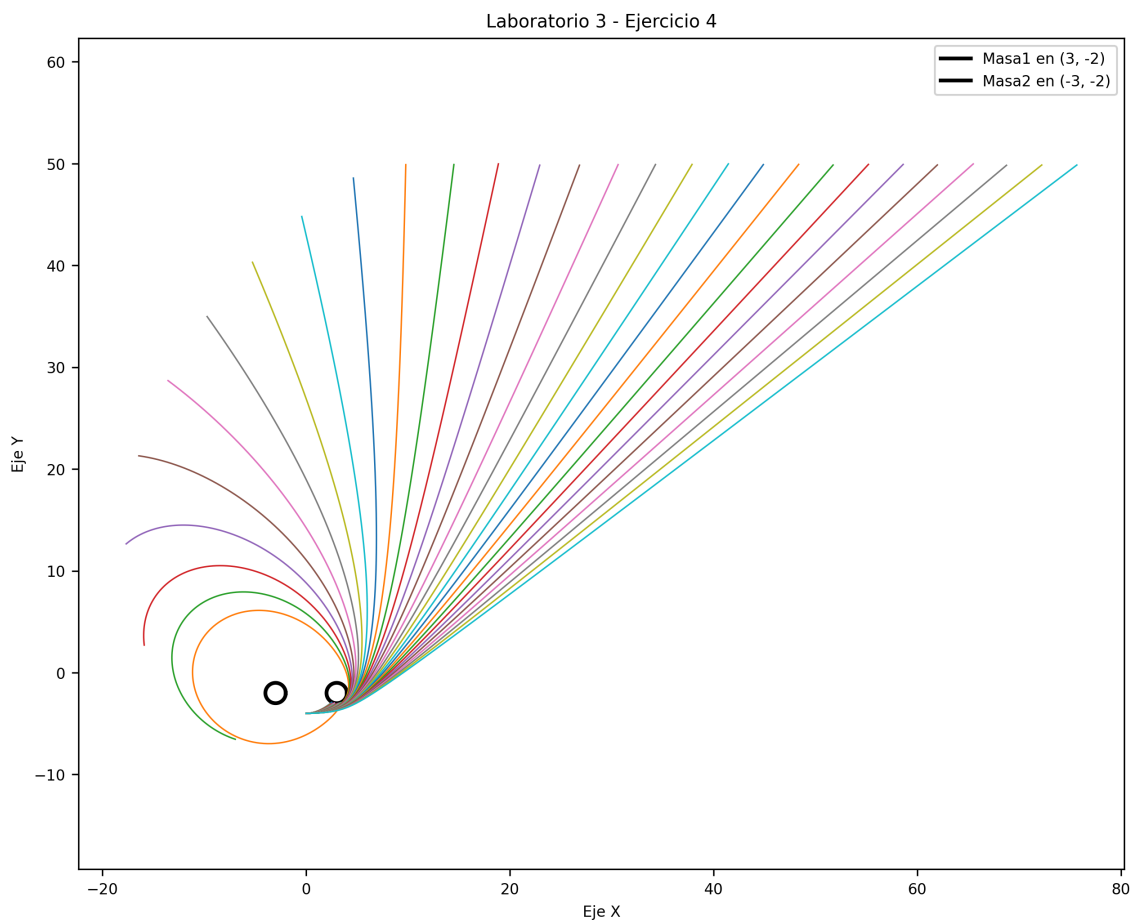


Figura 4: Gráfico de las trayectorias sin colapsar ni escapar

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

Resultados: El gráfico presenta únicamente trayectorias que permanecen ligadas al sistema de dos masas. Como se ve, hay un gran número de trayectorias que se abren radialmente hacia arriba sin llegar a alcanzar $y > 50$. Las órbitas más cerradas y oscilatorias alrededor de las masas son las que ni colapsan ni escapan, conformando una familia discreta de órbitas periódicas. Se observa una cuña de trayectorias que emergen del espacio entre las dos masas, demostrando que la geometría del sistema de dos cuerpos crea canales de movimiento preferentes. Las trayectorias largas y casi rectas indican movimiento de alta energía con deflexión controlada por ambas masas.

1.5 Ejercicio 5: 4 trayectorias

Problema: Encuentre 4 trayectorias

Implementación: Se buscan específicamente 4 trayectorias que no sufran colapso ni escape. El algoritmo itera sobre velocidades iniciales v_x usando interpolación lineal entre 0.01 y 3, y al encontrar una trayectoria válida, la almacena con etiqueta y contador. Se incrementa precisión mediante paso temporal más pequeño $h = 0.0025$ y tiempo de simulación prolongado $t_{\text{fin}} = 300$ segundos. Las condiciones de validez se mantienen: $|x|, |y| < 50$ y distancia a masas $> r = 1$. Una vez encontradas las 4 trayectorias, el bucle se detiene.

Código fuente: exe5.py

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({"font.size": 8, "lines.linewidth": 0.8})
plt.figure(figsize=(8, 8))

# Datos iniciales
r = 1.0
h = 0.0025
k = 0.01
tfin = 300

# Circunferencia 1
b1, c1 = 3, -2
theta = np.linspace(0, 2 * np.pi, 100)
x_circ = b1 + np.cos(theta)
y_circ = c1 + np.sin(theta)
plt.plot(x_circ, y_circ, "black", linewidth=2, label=f"Masa1 en ({b1}, {c1})")

# Circunferencia 2
b2, c2 = -3, -2
x_circ2 = b2 + np.cos(theta)
y_circ2 = c2 + np.sin(theta)
plt.plot(x_circ2, y_circ2, "black", linewidth=2, label=f"Masa2 en ({b2}, {c2})")

def ax(x, y, xc1, yc1, xc2, yc2):
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)
    if den1 == 0 or den2 == 0:
        return 0.0
    return -(x - xc1) / den1 - ((x - xc2) / den2)

def ay(x, y, xc1, yc1, xc2, yc2):
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)
    if den1 == 0 or den2 == 0:
        return 0.0
    return -(y - yc1) / den1 - ((y - yc2) / den2)

contador = 0

for vx0 in np.linspace(0.01, 3, int(3 / k)):
    if contador >= 4:
        break
    x, y = 0.0, 7.0
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```

vx, vy = vx0, 0

px = [x]
py = [y]

seEscapo = False
colapso = False

for t in np.arange(0, tfin, h):
    if (
        np.sqrt((x - b1) ** 2 + (y - c1) ** 2) <= r
        or np.sqrt((x - b2) ** 2 + (y - c2) ** 2) <= r
    ):
        colapso = True
        break

    if abs(y) > 50 or abs(x) > 50:
        seEscapo = True
        break

    vx += ax(x, y, b1, c1, b2, c2) * h
    vy += ay(x, y, b1, c1, b2, c2) * h
    x += vx * h
    y += vy * h

    px.append(x)
    py.append(y)

if not seEscapo and not colapso:
    plt.plot(px, py, label=f"Trayectoria {contador + 1}")
    contador += 1

# Configuración de la gráfica
plt.title("Laboratorio 3 - Ejercicio 5")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.grid(False)
plt.legend(loc="upper right")
plt.gca().set_aspect("equal", adjustable="box")
plt.savefig("img/pr_b/exe5.png", dpi=300, bbox_inches="tight")
plt.show()

```

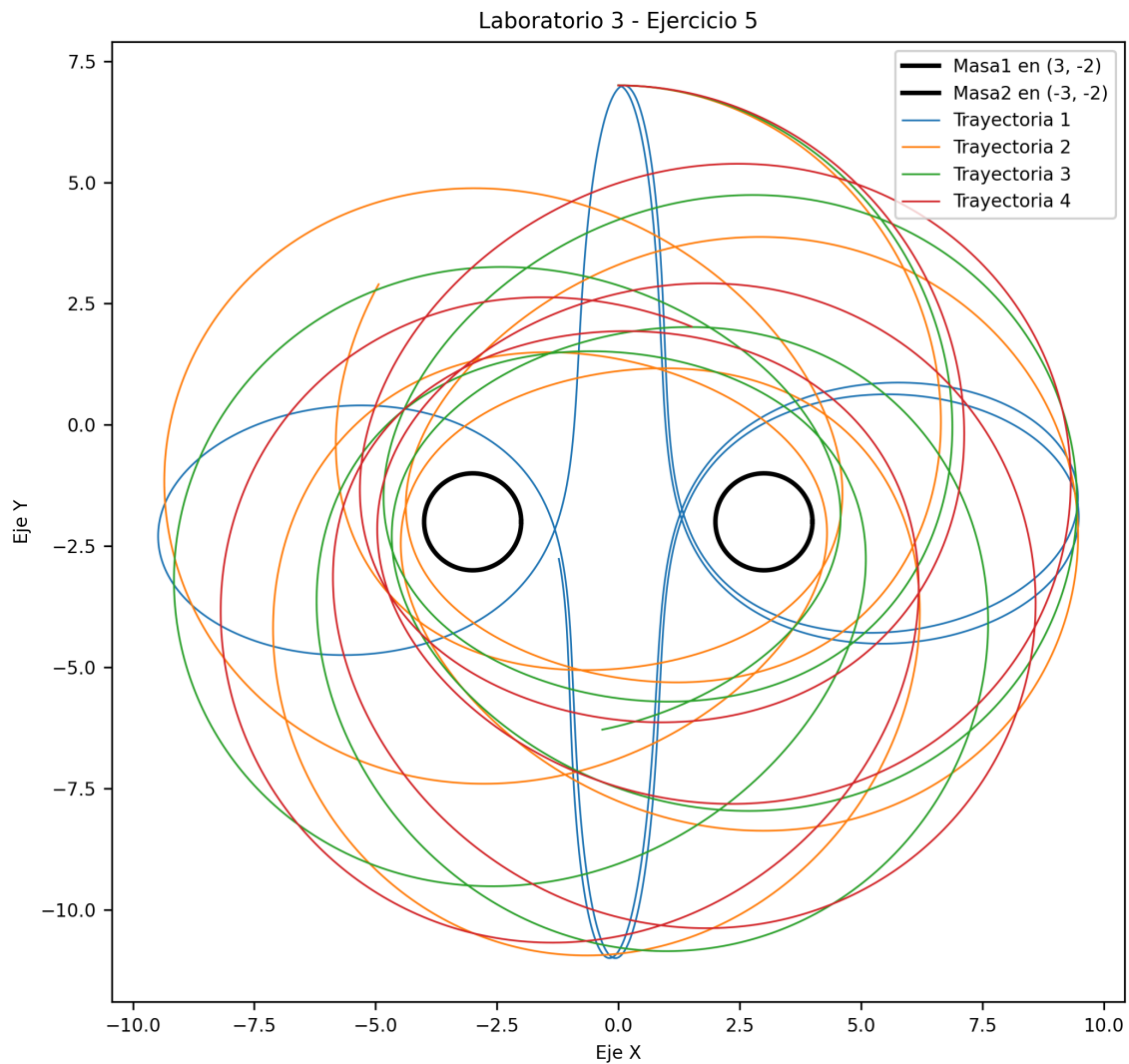


Figura 5: Gráfico de las 4 trayectorias

Resultados: El gráfico muestra 4 órbitas periódicas y quasiperiódicas alrededor del sistema de dos masas. Como se ve, cada trayectoria posee una firma característica: la trayectoria 1 (azul) ejecuta oscilaciones verticales pronunciadas cruzando entre ambas masas, la trayectoria 2 (naranja) forma órbitas más anchas alrededor del centro del sistema, la trayectoria 3 (verde) describe bucles cerrados sobre la derecha, y la trayectoria 4 (roja) muestra órbitas alargadas. Las cuatro trayectorias convergen cerca del punto de silla entre las masas, demostrando que existen canales de resonancia orbital. La simetría aproximada respecto al eje Y sugiere que el sistema de dos masas equidistantes del origen crea potencial simétrico con familia de órbitas predecibles.

1.6 Ejercicio 6: Trayectorias y Naves

Problema: Una nave se ubica (x, y) y otra en $(x + 0.01, y)$. Si ambas tienen las mismas iniciales, dibuje solo dos trayectorias que correspondan a las dos naves. Considere tiempos largos. Que observa?

Implementación: Se implementan dos naves idénticas salvo en posición inicial: Nave A en $(0, 7)$ y Nave B en $(0.01, 7)$, separadas por $\Delta x = 0.01$. Ambas tienen velocidad inicial $v_x = 8 \times 10^{-7}$, $v_y = 10^{-8}$ (valores cercanos a cero para observar la trayectoria). El paso de tiempo es $h = 0.01$ y la simulación corre durante $t_{\text{fin}} = 800$ segundos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

Lógica del Ejercicio 6 - Explicación Detallada: La clave de este ejercicio radica en entender que el parámetro que influencia el comportamiento es principalmente v_x , no las posiciones iniciales. Aunque se establece una pequeña separación de 0.01 unidades, el valor crítico es que v_x sea cercano a cero. Esta es la razón por la que se puede observar claramente la trayectoria: con v_x pequeño, la partícula no escapa rápidamente del sistema y permanece lo suficiente bajo influencia gravitacional para describir órbitas complejas. Si v_x fuera grande, ambas naves atravesarían rápidamente el sistema sin ser capturadas. Con v_x pequeño, la nave desacelera en la región de las masas, se vuelve en una órbita, y desarrolla una dinámica sensible a pequeñas perturbaciones. La separación inicial de 0.01 es suficientemente pequeña que las dos naves siguen trayectorias prácticamente idénticas al inicio, pero la aceleración no lineal (dependencia proporcional a distancia inversa al cubo) amplifica la diferencia exponencialmente en tiempos largos, generando divergencia caótica.

Código fuente: exe6.py

```
import matplotlib.pyplot as plt
import numpy as np

# Configuración inicial
plt.figure(figsize=(10, 8))

# Datos iniciales
r = 1.0

h = 0.01
tfin = 800
x0, y0 = 0.0, 7.0
vx0, vy0 = 0.00000080, 0.00000001

# Posiciones de las masas
b1, c1 = 3, -2
b2, c2 = -3, -2

# Dibujar masas
theta = np.linspace(0, 2 * np.pi, 100)
plt.plot(b1 + np.cos(theta), c1 + np.sin(theta), "black", label="Masa 1")
plt.plot(b2 + np.cos(theta), c2 + np.sin(theta), "black", label="Masa 2")

def ax(x, y, xc1, yc1, xc2, yc2):
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)
    if den1 == 0 or den2 == 0:
        return 0.0
    return -(x - xc1) / den1 - ((x - xc2) / den2)

def ay(x, y, xc1, yc1, xc2, yc2):
    den1 = ((x - xc1) ** 2 + (y - yc1) ** 2) ** (3 / 2)
    den2 = ((x - xc2) ** 2 + (y - yc2) ** 2) ** (3 / 2)
    if den1 == 0 or den2 == 0:
        return 0.0
    return -(y - yc1) / den1 - ((y - yc2) / den2)

# Definimos las dos naves
naves = [
    {"x": x0, "y": y0, "color": "blue", "label": "Nave A (x)"},
    {"x": x0 + 0.01, "y": y0, "color": "red", "label": "Nave B (x + 0.01)"},
]

for nave in naves:
    x, y = nave["x"], nave["y"]
    vx, vy = vx0, vy0
    tray_x, tray_y = [x], [y]

    for t in np.arange(0, tfin, h):
        acx = ax(x, y, b1, c1, b2, c2)
        acy = ay(x, y, b1, c1, b2, c2)
        vx += acx * h
        vy += acy * h
        x += vx * h
```

```

y += vy * h
tray_x.append(x)
tray_y.append(y)

dist1 = np.sqrt((x - b1) ** 2 + (y - c1) ** 2)
dist2 = np.sqrt((x - b2) ** 2 + (y - c2) ** 2)

if dist1 <= r or dist2 <= r:
    break

plt.plot(tray_x, tray_y, color=nave["color"], label=nave["label"], alpha=0.8)

plt.title("Ejercicio 6: Trayectorias y naves")
plt.legend()
plt.grid(True)
plt.gca().set_aspect("equal")
plt.savefig("img/pr_b/exe6.png", dpi=300)
plt.show()

```

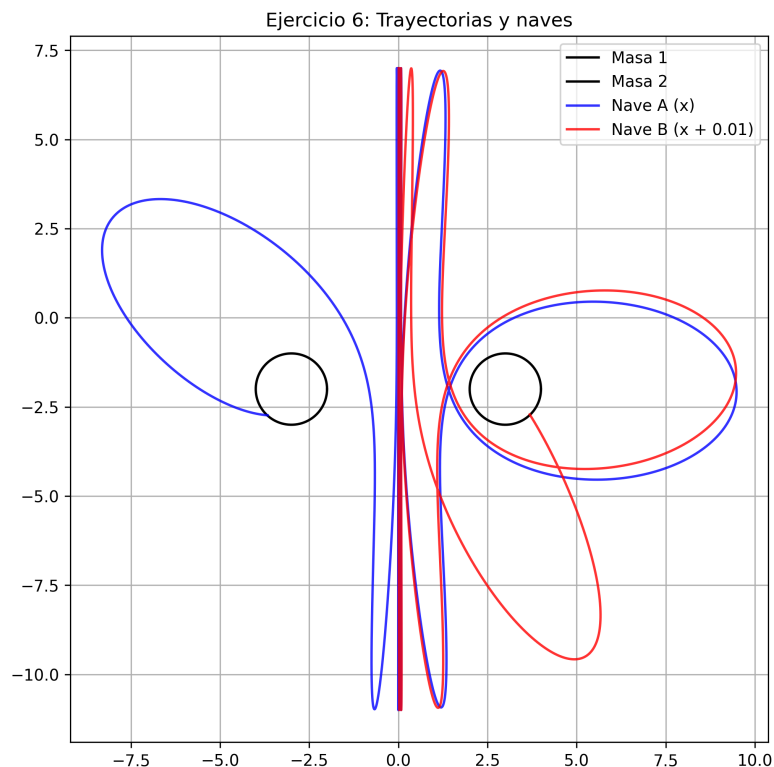


Figura 6: Gráfico de las trayectorias de las dos naves

Resultados: El gráfico muestra un fenómeno de caos determinista. Como se ve, ambas naves (azul y rojo) comienzan superpuestas y siguen trayectorias casi idénticas durante los primeros ciclos orbitales alrededor de las dos masas. Las naves orbitan juntas alrededor del punto de equilibrio entre las dos masas en (3, -2) y (-3, -2), ejecutando oscilaciones en espiral. Sin embargo, después de varios ciclos, las trayectorias divergen significativamente: la nave A (azul) forma un bucle más cerrado a la derecha, mientras que la nave B (rojo) desarrolla una órbita más abierta hacia abajo. Esta divergencia exponencial es característica de

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

sistemas caóticos donde pequeñas diferencias iniciales se amplifican por la naturaleza no lineal de las fuerzas gravitacionales. El hecho de que inicialmente sean prácticamente indistinguibles pero terminen muy separadas demuestra la sensibilidad a condiciones iniciales, marca fundamental de la dinámica caótica. La observación es que el sistema de tres cuerpos (las dos masas y la nave) exhibe comportamiento impredecible a largo plazo, validando que perturbaciones minúsculas producen trayectorias radicalmente diferentes.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. El problema de dos y tres cuerpos en gravitación exhibe comportamiento fundamentalmente distinto del problema de un cuerpo. Mientras que una sola masa central produce órbitas predecibles (elipses, parábolas, hipérbolas), dos masas generan dinámica caótica con órbitas periódicas alternadas con movimiento impredecible. Los ejercicios 1-5 demuestran cómo la arquitectura orbital se vuelve progresivamente más compleja: del sistema simple de una masa (ej. 1) a la complejidad de dos masas (ej. 2-5), donde coexisten órbitas ligadas, escape gravitacional, y resonancias periódicas.
2. El ejercicio 6 demuestra la sensibilidad extrema a condiciones iniciales característica de sistemas caóticos. Dos naves separadas por apenas 0.01 unidades en posición x , siguiendo ecuaciones de movimiento idénticas y deterministas, divergen completamente después de tiempos prolongados. Esto prueba que la integración numérica captura la esencia del caos determinista: predictibilidad a corto plazo con impredecibilidad a largo plazo. El papel crucial de v_x cercano a cero permite que las perturbaciones exponenciales de la dinámica no lineal se desarrollen completamente antes del escape del sistema.

RECOMENDACIONES

1. Para futuras prácticas, se recomienda explorar la influencia de la masa relativa de las dos masas centrales en la dinámica orbital. Variar la relación de masas (por ejemplo, una masa mucho mayor que la otra) podría revelar cómo el sistema transita entre comportamiento más predecible (dominancia de una masa) a caos completo (masas comparables). Además, implementar métodos de integración numérica más avanzados (como Runge-Kutta de orden superior) podría mejorar la precisión y permitir simular trayectorias durante tiempos aún más largos sin pérdida significativa de energía, lo que es crucial para estudiar el caos a largo plazo.